Knowledge Distillation and Cloud Deployment of a Lightweight Command-Parsing Language Model

CS3081 Cloud Computing Mini Project

Student: Muhammad Nehal
Repository: github.com/imnehal360/cs3081-cloud-mini-project
Date: 9 November 2026

Abstract

This project studies sequence-level knowledge distillation for a structured command-parsing task. A phi3.5 teacher generated labels that were cleaned and used to fine-tune the compact EleutherAI/pythia-70m student. Two student checkpoints were evaluated on 100 validation examples. A FastAPI inference service was containerized and deployed to k3d Kubernetes on GitHub-hosted runners. Automated tests, rolling deployment experiments, replica recovery, load distribution and Locust measurements provide evidence of functionality. The report distinguishes completed measurements from planned or incomplete infrastructure requirements.

Key measured findings

The cleaning stage retained 2,763 of 3,000 teacher outputs. Student v1 attained 100% syntactically valid JSON, 83% intent accuracy and 40% complete-frame accuracy on 100 validation examples. A four-user, two-minute Locust run completed 365 requests with zero recorded errors, 1.3 s p50, 2.9 s p95 and 3.07 requests/s.

Scope and reproducibility note

Teacher labeling and Kubernetes execution were performed on remote GitHub Actions runners; training used a Google Colab T4 GPU rather than the local Mac. This is a practical deviation from a fully local-cloud or CPU-only workflow, not evidence of compliance with those specific environment constraints.

1. Architecture and data pipeline

Logical architecture: source command pool -> deterministic teacher labeling (phi3.5, temperature 0, seed 42) -> two saved shards -> cleaning, schema checks and deduplication -> 1,500 training examples -> Pythia-70M student -> validation -> FastAPI -> Docker -> Kubernetes ingress and service.

Two parallel labeling workers generated 1,500 rows each. Raw outputs were saved as shard files, permitting resumption. The teacher was prompted to return only a JSON object containing the four requested fields: intent, date, time and place.

1.1 Cleaning results

Category	Count
Teacher outputs	3,000
Invented slot excluded	154
Bad intent excluded	28
Duplicates excluded	48
Wrong-key records excluded	7
Invalid JSON	0
Retained records	2,763

Clean dataset SHA-256: 69e7efb50903f90f2eb995ab534fe0fe60245191bb4e87afdddb340519bfa027

1.2 Training procedure

A decoder-only Pythia-70M student was fine-tuned on the first 1,500 cleaned examples. Input was formatted as Command: <input> followed by JSON: <teacher output>. The training loss masked the instruction prefix and padding, focusing the cross-entropy objective on JSON completion tokens. The maximum sequence length was 128, batch size 8, learning rate 5e-5 and training duration 3 epochs, with mixed precision on a Colab T4.

Variant v1 used seed 42; variant v2 used seed 7. This is sequence-level distillation because the student learns teacher-generated textual answers rather than matching teacher probability logits. The original teacher probabilities were not stored, so logit-based distillation was not performed.

Checkpoint	Epoch 1 loss	Epoch 2 loss	Epoch 3 loss
v1	0.4305	0.1197	0.0530
v2	0.4396	0.1330	0.0591

2. Model evaluation and limitations

Evaluation used 100 labeled validation commands and the project scoring script. The table reports independently measured student metrics. Teacher and untrained-base comparison scores were not available in the recorded evidence and are not inferred here.

Metric	Student v1	Student v2
JSON validity	100%	100%
Intent accuracy	83%	79%
Date accuracy	80%	80%
Time accuracy	80%	85%
Place accuracy	79%	79%
Complete-frame accuracy	40%	39%

Although both checkpoints produced syntactically valid JSON in this validation set, slot accuracy and complete-frame accuracy were substantially lower. Structured validity alone does not establish correct semantic parsing. Student v1 was therefore retained as the primary deployed model on these observed scores.

2.1 Artifact management

Student v1 and v2 checkpoints were distributed as ZIP archives with SHA-256 files under the GitHub Release tag model-v1; the trained weights were not intended for direct Git history. Archive digests were:

v1: abb8e05d1711a1b1cafb24901667db720d730f29cc687ba3ff835c426cac2b8d
v2: b671b54d2fa631433754ef800e528206c4cd726c0840f842fe7a57a0e446674e

3. Inference service and container

FastAPI serves POST /parse for command parsing, GET /healthz for liveness and GET /readyz for readiness. The response is a four-field JSON object. The service identifies its serving pod and model version in HTTP headers, and invalid model output is represented as HTTP 500 with the object {"error":"invalid_model_output"}. Model weights are fetched at startup by the deployed application rather than baked into the container image.

The image uses a CPU PyTorch runtime. The tested Kubernetes container limits were 1 vCPU and 1 GiB RAM per pod, with requests of 250 millicores and 512 MiB. The model archive download is from the public GitHub Release; the required SeaweedFS/S3 model-download path was not completed.

4. Kubernetes deployment and resilience

A k3d cluster was provisioned in GitHub Actions. The application namespace used a two-replica Deployment, a ClusterIP Service and Traefik Ingress exposing the API on localhost port 8080. Readiness and liveness probes targeted /readyz and /healthz.

Configuration	Observed value
Replicas	2 ready pods
CPU request / limit	250m / 1000m per pod
Memory request / limit	512Mi / 1Gi per pod
Ingress	Traefik
HPA target	70% CPU; min 1, max 2

Autoscaling: during a manual CPU-stress step the HPA increased from one replica to two; the earlier log indicated approximately 400% relative CPU utilization against a 70% target. The scale-down portion was not conclusively observed. A later evidence capture reported the HPA object missing because it had been removed before the rolling-update stage.

Rolling update and rollback: modifying MODEL_VERSION to v2 and waiting for rollout produced valid responses identifying v2. A rollback restored responses identifying v1. These checks passed with retry logic, but the experiment did not measure failure rate under continuous concurrent load and did not verify changing MODEL_URI.

Pod recovery: deleting a pod and waiting for the Deployment showed restoration to 2/2 ready pods. This verifies eventual reconciliation, but the exact under-load recovery error rate was not measured.

4.1 Load test

A Locust run used four virtual users for two minutes against POST /parse. The reported summary was:

Measurement	Result
Requests / failed	365 / 0
Failure rate	0.0%
Median (p50)	1,300 ms
p95	2,900 ms
Average	1,300 ms
Throughput	3.067 requests/s
Fastest / slowest	375 ms / 3,455 ms

The recorded per-pod request distribution was 181 and 185 observations (49.45% and 50.55%). Its total of 366 header observations differs from the Locust count of 365; therefore these figures demonstrate approximate balance but should not be treated as reconciled request totals. Both pods exceeded the 30% traffic-share threshold.

5. Network design and automation

The 10.20.0.0/16 address range was allocated by VLSM, largest requirement first, with non-overlapping contiguous networks.

Zone	Required hosts	CIDR	Capacity
app	1,000	10.20.0.0/22	1,022
public	250	10.20.4.0/24	254
data	100	10.20.5.0/25	126
management	20	10.20.5.128/27	30

The public-subnet NACL denies the 198.51.100.0/24 source range before allow rules, permits inbound TCP 80/443 from anywhere, TCP 22 only from management, response ephemeral ports to the app subnet, outbound TCP 8000 to app and ephemeral replies to clients. The provided NACL simulator accepted all six supplied sample packets (6/6); hidden-case results are unknown.

5.1 CI and quality controls

A successful GitHub Actions pipeline performed Ruff linting, FastAPI unit tests, model release download and checksum checking, validation scoring with a minimum 95% JSON-validity gate, and Gitleaks secret scanning. A deliberate negative evaluation on 100 malformed predictions produced 0% validity and the expected QUALITY GATE FAILED outcome. This shows that the quality gate rejects malformed predictions rather than merely reporting a score.

6. Discussion, cost and next improvements

Under measured four-user load the application recorded no failed requests and p95 of 2.9 seconds. These results apply only to the tested runner, configuration, traffic pattern and test duration. They are not a general service-level guarantee. The CPU and memory limits are known, but a verified resource-usage time series and rate-card based cost total were not available; a numerical cloud-cost estimate is therefore intentionally omitted rather than fabricated.

Outstanding work includes SeaweedFS/S3 integration for labels and weights, teacher and untrained-base evaluation, repeatable training Make targets and archived environment specifications, automated three-run cold-start timings, verified HPA scale-down, under-load rolling/rollback and pod-failure error rates, rate-card cost calculation, and formal validation of the container image size. These gaps should be addressed before claiming full specification compliance.

7. Conclusion

This implementation demonstrates that a small language model can learn a structured JSON parsing format from teacher-generated command examples and be served via a containerized, health-checked Kubernetes API. The strongest measured outcomes are 100% JSON validity on validation, automated release-based CI, 0% Locust errors in the recorded two-minute run, and approximately even traffic distribution across two pods. Complete-frame accuracy and unfinished object-storage and experiment requirements remain the principal limitations.

Reference: Source code, workflow definitions, training and evaluation artifacts:
https://github.com/imnehal360/cs3081-cloud-mini-project